# SkillSync HRMS

## Backend Implementation Guide

Node.js · Express · PostgreSQL · Supabase

Full-Stack Integration Reference

Generated: 13 April 2026

# 1. Architecture Overview

SkillSync's backend replaces the current client-side mockData.ts with a real Node.js/Express API layer backed by PostgreSQL (via Supabase). The frontend Flutter sends …requests to the API; no scoring logic lives in the browser.

## 1.1 High-Level Stack

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | React 18 + TypeScript + Vite | SPA served from CDN or static host |
| API Server | Node.js 20 LTS + Express 5 | REST API, business logic, algorithmic scoring |
| ORM / Query | Drizzle ORM + pg driver | Type-safe SQL queries against PostgreSQL |
| Database | PostgreSQL 15 (Supabase) | Primary relational data store |
| Auth | Supabase Auth (JWT) | Session tokens validated by API middleware |
| File Storage | Supabase Storage | Avatars, policy PDFs, payslips |
| Real-time | Supabase Realtime | Chat messages, notifications pushed to clients |
| Cache | Redis (optional) | Rate limiting, session cache, scoring cache |
| Task Queue | BullMQ + Redis | Background jobs: payroll runs, email notifications |

## 1.2 Request Flow

```
Browser (React)
    │  HTTP/HTTPS + Authorization: Bearer <supabase-jwt>
    ▼
Express API  (api.skillsync.internal)
    │  verifyToken()  ──▶  Supabase Auth  (validate JWT, get user_id+role)
    │  authorizeRole() ──▶ check app_role enum in user_roles table
    │
    ├── /api/employees    ──▶  Employee service  ──▶  PostgreSQL
    ├── /api/skills       ──▶  Skill service     ──▶  PostgreSQL
    ├── /api/turnover     ──▶  Analytics service ──▶  PostgreSQL
    ├── /api/chat         ──▶  Chat service      ──▶  PostgreSQL + Realtime
    └── /api/payroll      ──▶  Payroll service   ──▶  PostgreSQL + BullMQ
```

# 2. Project Structure

```
skillsync-api/
├── src/
│   ├── index.ts              # Entry point, server bootstrap
│   ├── app.ts                # Express app factory
│   ├── config/
│   │   ├── env.ts            # Zod-validated environment variables
│   │   └── database.ts       # Drizzle + pg pool setup
│   ├── middleware/
│   │   ├── auth.ts           # JWT verification via Supabase
│   │   ├── authorize.ts      # Role-based access control
│   │   ├── errorHandler.ts   # Global error boundary
│   │   └── requestLogger.ts  # Morgan / pino logging
│   ├── db/
│   │   ├── schema/           # Drizzle table definitions
│   │   ├── migrations/       # Generated SQL migration files
│   │   └── seed.ts           # Seed script (imports mockData.ts data)
│   ├── modules/              # Feature modules (one folder per domain)
│   │   ├── employees/        # routes, controller, service, repository
│   │   ├── skills/
│   │   ├── attendance/
│   │   ├── leaves/
│   │   ├── payroll/
│   │   ├── todos/
│   │   ├── notifications/
│   │   ├── chat/
│   │   ├── analytics/        # Turnover risk, skill gaps, workforce
│   │   ├── roles/
│   │   ├── departments/
│   │   ├── holidays/
│   │   ├── audit/
│   │   └── resignations/
│   ├── scoring/              # Algorithmic intelligence
│   │   ├── roleFit.ts        # calculateRoleFit()
│   │   ├── turnoverRisk.ts   # multi-factor risk scoring
│   │   ├── replacements.ts   # findReplacementCandidates()
│   │   └── skillGaps.ts      # getOrganizationSkillGaps()
│   ├── jobs/                 # BullMQ background workers
│   │   ├── payrollWorker.ts
│   │   └── notificationWorker.ts
│   └── utils/
│       ├── pagination.ts
│       └── apiResponse.ts
├── .env                      # Local secrets (never committed)
├── drizzle.config.ts
├── package.json
└── tsconfig.json
```

# 3. Environment Setup

## 3.1 Prerequisites

- Node.js 20 LTS — https://nodejs.org
- PostgreSQL 15 running locally or via Supabase cloud
- Redis 7 (optional but recommended for queues and caching)
- Supabase project created at https://supabase.com — copy the project URL and service-role key

## 3.2 Initialize the Project

```
mkdir skillsync-api && cd skillsync-api
npm init -y

# Core dependencies
npm install express drizzle-orm drizzle-kit pg @supabase/supabase-js
npm install zod express-validator jsonwebtoken
npm install bullmq ioredis
npm install morgan pino pino-http
npm install cors helmet express-rate-limit
npm install date-fns uuid

# Dev dependencies
npm install -D typescript @types/node @types/express @types/pg
npm install -D @types/jsonwebtoken @types/morgan @types/cors
npm install -D ts-node nodemon tsx
npm install -D dotenv
```

## 3.3 Environment Variables (.env)

```
# ── Server ──────────────────────────────────────────────
PORT=4000
NODE_ENV=development

# ── Database ────────────────────────────────────────────
DATABASE_URL=postgresql://postgres:<password>@db.<project>.supabase.co:5432/postgres

# ── Supabase ────────────────────────────────────────────
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_SERVICE_ROLE_KEY=<service-role-key>  # Keep secret - server only
SUPABASE_JWT_SECRET=<jwt-secret>              # From Supabase → Settings → API

# ── Redis (optional) ────────────────────────────────────
REDIS_URL=redis://localhost:6379

# ── Frontend ────────────────────────────────────────────
FRONTEND_URL=http://localhost:5173
```

> TIP: Use a package like dotenv-safe and commit a .env.example with all keys but no values, so new developers know exactly what they need to configure.

## 3.4 tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "paths": { "@/*": ["./src/*"] }
  },
  "include": ["src/**/*"],
```

```
  "exclude": ["node_modules", "dist"]
}
```

# 4. Database Schema (Drizzle ORM)

All tables are defined as Drizzle schema objects. Drizzle generates and manages the SQL migrations. The schema mirrors what Supabase already defined in supabase/types.ts, with additions for the ML scoring tables.

## 4.1 Schema File (src/db/schema/index.ts)

```
import { pgTable, pgEnum, uuid, text, integer, numeric,
         boolean, timestamp, date, jsonb, index } from 'drizzle-orm/pg-core';

// ── Enums ──────────────────────────────────────────────────────────────
export const appRoleEnum    = pgEnum('app_role', ['employee','manager','hr_admin']);
export const leaveTypeEnum  = pgEnum('leave_type', ['annual','sick','compassionate']);
export const leaveStatusEnum= pgEnum('leave_status', ['pending','approved','rejected']);
export const attendanceStatusEnum = pgEnum('attendance_status',
  ['present','absent','late','half_day','remote']);
export const priorityEnum   = pgEnum('priority', ['low','medium','high']);
export const skillCategoryEnum = pgEnum('skill_category',
  ['technical','soft','management','domain']);
export const roleLevelEnum  = pgEnum('role_level',
  ['junior','mid','senior','lead','manager']);
export const holidayTypeEnum= pgEnum('holiday_type',['public','company','optional']);
export const payrollStatusEnum = pgEnum('payroll_status',['pending','paid','failed']);

// ── profiles ───────────────────────────────────────────────────────────
export const profiles = pgTable('profiles', {
  id:         uuid('id').primaryKey().defaultRandom(),
  userId:     uuid('user_id').notNull().unique(),
  fullName:   text('full_name').notNull(),
  email:      text('email').notNull().unique(),
  phone:      text('phone'),
  department: text('department'),
  position:   text('position'),
  salary:     numeric('salary', { precision: 12, scale: 2 }),
  hireDate:   date('hire_date'),
  avatarUrl:  text('avatar_url'),
  createdAt:  timestamp('created_at').defaultNow(),
  updatedAt:  timestamp('updated_at').defaultNow(),
}, t => ({ userIdx: index('profiles_user_id_idx').on(t.userId) }));

// ── user_roles ─────────────────────────────────────────────────────────
export const userRoles = pgTable('user_roles', {
  id:     uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').notNull().unique(),
  role:   appRoleEnum('role').notNull().default('employee'),
});

// ── departments ────────────────────────────────────────────────────────
export const departments = pgTable('departments', {
  id:          uuid('id').primaryKey().defaultRandom(),
  name:        text('name').notNull().unique(),
```

```
  description: text('description'),
  managerId:   uuid('manager_id').references(() => profiles.id),
  createdAt:   timestamp('created_at').defaultNow(),
});

// ── job_roles ──────────────────────────────────────────────────────────
export const jobRoles = pgTable('job_roles', {
  id:             uuid('id').primaryKey().defaultRandom(),
  title:          text('title').notNull(),
  department:     text('department').notNull(),
  level:          roleLevelEnum('level').notNull(),
  description:    text('description'),
  salaryRangeMin: numeric('salary_range_min', { precision: 12, scale: 2 }),
  salaryRangeMax: numeric('salary_range_max', { precision: 12, scale: 2 }),
  createdAt:      timestamp('created_at').defaultNow(),
});

// ── skills ─────────────────────────────────────────────────────────────
export const skills = pgTable('skills', {
  id:           text('id').primaryKey(),   // e.g. 'js', 'react'
  name:         text('name').notNull(),
  category:     skillCategoryEnum('category').notNull(),
  description:  text('description'),
  relatedSkills: text('related_skills').array(),
  createdAt:    timestamp('created_at').defaultNow(),
});

// ── role_required_skills (join) ────────────────────────────────────────
export const roleRequiredSkills = pgTable('role_required_skills', {
  id:              uuid('id').primaryKey().defaultRandom(),
  roleId:          uuid('role_id').notNull().references(() => jobRoles.id),
  skillId:         text('skill_id').notNull().references(() => skills.id),
  minProficiency:  integer('min_proficiency').notNull(),
});

// ── employee_skills (join) ─────────────────────────────────────────────
export const employeeSkills = pgTable('employee_skills', {
  id:           uuid('id').primaryKey().defaultRandom(),
  profileId:    uuid('profile_id').notNull().references(() => profiles.id),
  skillId:      text('skill_id').notNull().references(() => skills.id),
  proficiency:  integer('proficiency').notNull().default(0), // 0-5
  lastAssessed: timestamp('last_assessed').defaultNow(),
});

// ── skill_chains ───────────────────────────────────────────────────────
export const skillChains = pgTable('skill_chains', {
  id:           uuid('id').primaryKey().defaultRandom(),
  fromSkillId:  text('from_skill_id').notNull().references(() => skills.id),
  toSkillId:    text('to_skill_id').notNull().references(() => skills.id),
  description:  text('description'),
});

// ── learning_items ─────────────────────────────────────────────────────
export const learningItems = pgTable('learning_items', {
  id:          uuid('id').primaryKey().defaultRandom(),
  title:       text('title').notNull(),
  description: text('description'),
  skillId:     text('skill_id').references(() => skills.id),
  type:        text('type').notNull(), // course|cert|project|mentorship
  duration:    text('duration'),
  priority:    priorityEnum('priority').notNull().default('medium'),
  url:         text('url'),
```

```
  createdAt:   timestamp('created_at').defaultNow(),
});

// ── attendance ──────────────────────────────────────────────────────────
export const attendance = pgTable('attendance', {
  id:        uuid('id').primaryKey().defaultRandom(),
  userId:    uuid('user_id').notNull().references(() => profiles.userId),
  date:      date('date').notNull(),
  checkIn:   text('check_in'),
  checkOut:  text('check_out'),
  status:    attendanceStatusEnum('status'),
  type:      text('type'),   // office | remote
  createdAt: timestamp('created_at').defaultNow(),
}, t => ({ userDate: index('attendance_user_date_idx').on(t.userId, t.date) }));

// ── leave_balances ──────────────────────────────────────────────────────
export const leaveBalances = pgTable('leave_balances', {
  id:        uuid('id').primaryKey().defaultRandom(),
  userId:    uuid('user_id').notNull().references(() => profiles.userId),
  leaveType: leaveTypeEnum('leave_type').notNull(),
  totalDays: integer('total_days').default(0),
  usedDays:  integer('used_days').default(0),
  year:      integer('year').notNull(),
});

// ── leave_requests ──────────────────────────────────────────────────────
export const leaveRequests = pgTable('leave_requests', {
  id:         uuid('id').primaryKey().defaultRandom(),
  userId:     uuid('user_id').notNull().references(() => profiles.userId),
  leaveType:  leaveTypeEnum('leave_type').notNull(),
  startDate:  date('start_date').notNull(),
  endDate:    date('end_date').notNull(),
  reason:     text('reason'),
  status:     leaveStatusEnum('status').default('pending'),
  approvedBy: uuid('approved_by').references(() => profiles.userId),
  createdAt:  timestamp('created_at').defaultNow(),
});

// ── payroll ─────────────────────────────────────────────────────────────
export const payroll = pgTable('payroll', {
  id:          uuid('id').primaryKey().defaultRandom(),
  userId:      uuid('user_id').notNull().references(() => profiles.userId),
  month:       integer('month').notNull(),
  year:        integer('year').notNull(),
  basicSalary: numeric('basic_salary', { precision: 12, scale: 2 }).notNull(),
  allowances:  numeric('allowances', { precision: 12, scale: 2 }).default('0'),
  deductions:  numeric('deductions', { precision: 12, scale: 2 }).default('0'),
  netSalary:   numeric('net_salary', { precision: 12, scale: 2 }).notNull(),
  status:      payrollStatusEnum('status').default('pending'),
  paidDate:    timestamp('paid_date'),
  createdAt:   timestamp('created_at').defaultNow(),
});

// ── todos ───────────────────────────────────────────────────────────────
export const todos = pgTable('todos', {
  id:          uuid('id').primaryKey().defaultRandom(),
  userId:      uuid('user_id').notNull().references(() => profiles.userId),
  title:       text('title').notNull(),
  description: text('description'),
  dueDate:     date('due_date'),
  priority:    priorityEnum('priority').default('medium'),
  completed:   boolean('completed').default(false),
```

```
  createdAt:   timestamp('created_at').defaultNow(),
});

// ── notifications ──────────────────────────────────────────────────────
export const notifications = pgTable('notifications', {
  id:        uuid('id').primaryKey().defaultRandom(),
  userId:    uuid('user_id').notNull().references(() => profiles.userId),
  title:     text('title').notNull(),
  message:   text('message').notNull(),
  type:      text('type'),
  read:      boolean('read').default(false),
  createdAt: timestamp('created_at').defaultNow(),
});

// ── chat_messages ──────────────────────────────────────────────────────
export const chatMessages = pgTable('chat_messages', {
  id:        uuid('id').primaryKey().defaultRandom(),
  senderId:  uuid('sender_id').notNull().references(() => profiles.userId),
  roomId:    text('room_id').notNull(), // e.g. 'hr-general', 'team-<id>'
  content:   text('content').notNull(),
  isBot:     boolean('is_bot').default(false),
  createdAt: timestamp('created_at').defaultNow(),
}, t => ({ roomIdx: index('chat_room_idx').on(t.roomId) }));

// ── resignation_requests ───────────────────────────────────────────────
export const resignationRequests = pgTable('resignation_requests', {
  id:               uuid('id').primaryKey().defaultRandom(),
  userId:           uuid('user_id').notNull().references(() => profiles.userId),
  lastWorkingDate:  date('last_working_date').notNull(),
  noticePeriodDays: integer('notice_period_days').default(30),
  reason:           text('reason'),
  status:           text('status').default('pending'),
  approvedBy:       uuid('approved_by').references(() => profiles.userId),
  createdAt:        timestamp('created_at').defaultNow(),
});

// ── holidays ───────────────────────────────────────────────────────────
export const holidays = pgTable('holidays', {
  id:          uuid('id').primaryKey().defaultRandom(),
  name:        text('name').notNull(),
  date:        date('date').notNull(),
  type:        holidayTypeEnum('type').notNull().default('public'),
  description: text('description'),
  createdAt:   timestamp('created_at').defaultNow(),
});

// ── audit_logs ─────────────────────────────────────────────────────────
export const auditLogs = pgTable('audit_logs', {
  id:         uuid('id').primaryKey().defaultRandom(),
  userId:     uuid('user_id').references(() => profiles.userId),
  action:     text('action').notNull(),
  entityType: text('entity_type').notNull(),
  entityId:   text('entity_id'),
  oldValues:  jsonb('old_values'),
  newValues:  jsonb('new_values'),
  ipAddress:  text('ip_address'),
  createdAt:  timestamp('created_at').defaultNow(),
});

// ── turnover_risk_cache ────────────────────────────────────────────────
// Stores pre-computed risk scores so the HR dashboard loads fast
export const turnoverRiskCache = pgTable('turnover_risk_cache', {
```

```
  id:           uuid('id').primaryKey().defaultRandom(),
  profileId:    uuid('profile_id').notNull().references(() => profiles.id),
  riskScore:    integer('risk_score').notNull(),
  riskLevel:    text('risk_level').notNull(),
  factors:      text('factors').array(),
  computedAt:   timestamp('computed_at').defaultNow(),
});
```

### 4.2 Migrations

```
# drizzle.config.ts
import type { Config } from 'drizzle-kit';
export default {
  schema: './src/db/schema/index.ts',
  out: './src/db/migrations',
  driver: 'pg',
  dbCredentials: { connectionString: process.env.DATABASE_URL! },
} satisfies Config;

# Generate migration SQL
npx drizzle-kit generate:pg

# Push directly to DB (dev only)
npx drizzle-kit push:pg

# Run migrations in production
npx drizzle-kit migrate
```

## 5. Authentication & Authorization Middleware

### 5.1 JWT Verification (src/middleware/auth.ts)

Supabase issues a signed JWT when the user logs in via the React app. The API validates this token on every request using the Supabase JWT secret — no extra round-trip to Supabase needed.

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { db } from '../config/database';
import { userRoles } from '../db/schema';
import { eq } from 'drizzle-orm';

export interface AuthRequest extends Request {
  user?: { id: string; email: string; role: string };
}

export async function verifyToken(req: AuthRequest, res: Response, next: NextFunction) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Missing token' });
  }
  const token = authHeader.slice(7);
  try {
    const payload = jwt.verify(token, process.env.SUPABASE_JWT_SECRET!) as any;
    // Fetch role from user_roles table (single query, cached by pg pool)
    const [row] = await db.select({ role: userRoles.role })
```

```
      .from(userRoles)
      .where(eq(userRoles.userId, payload.sub))
      .limit(1);
    req.user = { id: payload.sub, email: payload.email, role: row?.role ?? 'employee' };
    next();
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

### 5.2 Role Authorization (src/middleware/authorize.ts)

```
import { AuthRequest } from './auth';
import { Response, NextFunction } from 'express';

type AppRole = 'employee' | 'manager' | 'hr_admin';

export function authorize(...roles: AppRole[]) {
  return (req: AuthRequest, res: Response, next: NextFunction) => {
    if (!req.user || !roles.includes(req.user.role as AppRole)) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }
    next();
  };
}

// Usage in routes:
// router.get('/turnover', verifyToken, authorize('hr_admin'), getTurnoverRisk);
// router.get('/team',     verifyToken, authorize('manager','hr_admin'), getTeam);
```

## 6. Core API Modules

Each feature module follows a four-layer pattern: Router → Controller → Service → Repository. This keeps routing thin, business logic in services, and database queries isolated in repositories.

### 6.1 Employees Module

#### Routes (src/modules/employees/routes.ts)

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/employees | hr_admin | List all employees (paginated, filterable) |
| GET | /api/employees/:id | hr_admin \| manager | Get single employee + skills |
| POST | /api/employees | hr_admin | Create employee (profile + user_roles) |
| PATCH | /api/employees/:id | hr_admin | Update profile fields |
| DELETE | /api/employees/:id | hr_admin | Soft-delete (set status=inactive) |

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/employees/me | any | Get own profile |
| PATCH | /api/employees/me | any | Update own profile |

### Service (src/modules/employees/service.ts)

```
import { db } from '../../config/database';
import { profiles, employeeSkills, skills } from '../../db/schema';
import { eq, ilike, and } from 'drizzle-orm';

export async function listEmployees({ page = 1, limit = 20, department, search }: {
  page?: number; limit?: number; department?: string; search?: string;
}) {
  const offset = (page - 1) * limit;
  const conditions = [];
  if (department) conditions.push(eq(profiles.department, department));
  if (search)     conditions.push(ilike(profiles.fullName, `%${search}%`));

  const rows = await db
    .select()
    .from(profiles)
    .where(conditions.length ? and(...conditions) : undefined)
    .limit(limit)
    .offset(offset)
    .orderBy(profiles.fullName);

  return rows;
}

export async function getEmployeeWithSkills(profileId: string) {
  const [employee] = await db.select().from(profiles)
    .where(eq(profiles.id, profileId)).limit(1);
  if (!employee) return null;

  const empSkills = await db
    .select({ skillId: employeeSkills.skillId,
              proficiency: employeeSkills.proficiency,
              name: skills.name, category: skills.category })
    .from(employeeSkills)
    .innerJoin(skills, eq(employeeSkills.skillId, skills.id))
    .where(eq(employeeSkills.profileId, profileId));

  return { ...employee, skills: empSkills };
}
```

## 6.2 Skills Module

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/skills | any | All skills (with related skills) |
| GET | /api/skills/chains | any | Skill progression chains |
| GET | /api/skills/learning | any | Learning items (filterable by skillId, priority) |

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/skills/me | any | Own employee skill proficiencies |
| PATCH | /api/skills/me/:skillId | any | Self-update skill proficiency |
| POST | /api/skills | hr_admin | Create new skill |
| PATCH | /api/skills/:id | hr_admin | Edit skill |
| GET | /api/skills/gaps/org | hr_admin | Org-level skill gap analysis |

## 6.3 Attendance Module

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | /api/attendance/check-in | any | Record check-in for today |
| PATCH | /api/attendance/check-out | any | Record check-out |
| GET | /api/attendance/me | any | Own attendance history |
| GET | /api/attendance/team | manager | Team attendance (last 30 days) |
| GET | /api/attendance | hr_admin | Org-wide attendance |
| PATCH | /api/attendance/:id | hr_admin | Correct a record |

### Check-in Logic

```
export async function checkIn(userId: string) {
  const today = new Date().toISOString().split('T')[0];
  const existing = await db.select().from(attendance)
    .where(and(eq(attendance.userId, userId), eq(attendance.date, today)));

  if (existing.length) throw new Error('Already checked in today');

  const now = new Date();
  const checkInTime = now.toTimeString().slice(0,5); // 'HH:MM'
  const status = checkInTime > '09:00' ? 'late' : 'present';

  return db.insert(attendance).values({
    userId, date: today, checkIn: checkInTime,
    status, type: 'office',
  }).returning();
}
```

## 6.4 Leave Requests Module

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/leaves/balance | any | Own leave balance (all types) |
| POST | /api/leaves | any | Submit leave request |

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/leaves/me | any | Own requests |
| GET | /api/leaves/team | manager | Team pending requests |
| PATCH | /api/leaves/:id/approve | manager\|hr_admin | Approve (deducts from balance) |
| PATCH | /api/leaves/:id/reject | manager\|hr_admin | Reject with comment |
| GET | /api/leaves | hr_admin | All requests |

### Approval Service Logic

```
export async function approveLeave(requestId: string, approverId: string) {
  const [req] = await db.select().from(leaveRequests)
    .where(and(eq(leaveRequests.id, requestId),
               eq(leaveRequests.status, 'pending')));
  if (!req) throw new Error('Request not found or already processed');

  // Calculate working days between start and end date
  const days = countWorkingDays(req.startDate, req.endDate);

  // Check balance
  const [balance] = await db.select().from(leaveBalances)
    .where(and(eq(leaveBalances.userId, req.userId),
               eq(leaveBalances.leaveType, req.leaveType)));
  const available = (balance.totalDays ?? 0) - (balance.usedDays ?? 0);
  if (days > available) throw new Error('Insufficient leave balance');

  // Transaction: update request + deduct balance + create notification
  await db.transaction(async tx => {
    await tx.update(leaveRequests)
      .set({ status: 'approved', approvedBy: approverId })
      .where(eq(leaveRequests.id, requestId));
    await tx.update(leaveBalances)
      .set({ usedDays: sql`used_days + ${days}` })
      .where(eq(leaveBalances.id, balance.id));
    await tx.insert(notifications).values({
      userId: req.userId,
      title: 'Leave Approved',
      message: `Your ${req.leaveType} leave was approved.`,
    });
  });
}
```

## 6.5 Payroll Module

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/payroll/me | any | Own payslips (all months) |
| GET | /api/payroll/:userId | hr_admin | Any employee's payslips |
| GET | /api/payroll | hr_admin | All payroll records (month/year filter) |
| POST | /api/payroll/run | hr_admin | Trigger payroll run for given month/year |

| Method | Path | Auth | Description |
|---|---|---|---|
| PATCH | /api/payroll/:id/mark-paid | hr_admin | Mark record as paid + set paid_date |

> Payroll runs as a background BullMQ job. The POST /api/payroll/run endpoint enqueues the job and immediately returns { jobId }. The client polls GET /api/payroll/run/:jobId/status for progress.

### 6.6 Chat Module (Real-time)

Messages are stored in PostgreSQL and broadcast via Supabase Realtime. The React client subscribes to the Supabase channel directly for live updates.

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /api/chat/rooms | any | List chat rooms user belongs to |
| GET | /api/chat/rooms/:roomId | any | Message history (paginated, newest first) |
| POST | /api/chat/rooms/:roomId | any | Send message (written to DB → Realtime broadcasts) |

#### Real-time Setup (Supabase Realtime)

```
// In the React client (replaces the random response simulation)
import { createClient } from '@supabase/supabase-js';
const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

const channel = supabase
  .channel(`chat:${roomId}`)  // roomId e.g. 'hr-general', 'team-abc123'
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'chat_messages',
    filter: `room_id=eq.${roomId}`,
  }, payload => {
    setMessages(prev => [...prev, payload.new as Message]);
  })
  .subscribe();

return () => supabase.removeChannel(channel); // cleanup on unmount
```

## 7. Algorithmic Scoring Engine

All scoring functions move from mockData.ts to the API server. They now query real PostgreSQL data and return results as JSON. Results can be cached in the turnover_risk_cache table.

### 7.1 Role Fit Score (src/scoring/roleFit.ts)

```
import { db } from '../config/database';
```

```
import { employeeSkills, roleRequiredSkills } from '../db/schema';
import { eq } from 'drizzle-orm';

export async function calculateRoleFit(profileId: string, roleId: string) {
  const [required, actual] = await Promise.all([
    db.select().from(roleRequiredSkills).where(eq(roleRequiredSkills.roleId, roleId)),
    db.select().from(employeeSkills).where(eq(employeeSkills.profileId, profileId)),
  ]);

  let totalScore = 0, maxScore = 0;
  for (const req of required) {
    const emp = actual.find(s => s.skillId === req.skillId);
    const proficiency = emp?.proficiency ?? 0;
    totalScore += Math.min(proficiency, req.minProficiency);
    maxScore   += req.minProficiency;
  }
  return maxScore > 0 ? Math.round((totalScore / maxScore) * 100) : 0;
}

// Route:  GET /api/skills/fit?roleId=<uuid>
// Returns: { roleFit: 78, matchingSkills: [...], missingSkills: [...] }
```

## 7.2 Turnover Risk Score (src/scoring/turnoverRisk.ts)

Replaces the inline generateTurnoverRiskData() in the HR page. Reads real attendance, leave, and profile data from PostgreSQL.

```
export async function computeTurnoverRisk(profileId: string) {
  // Fetch all data in parallel
  const [profile, attendanceRows, leaveRows, roleRow] = await Promise.all([
    db.select().from(profiles).where(eq(profiles.id, profileId)).limit(1),
    db.select().from(attendance).where(and(
      eq(attendance.userId, profiles.userId),
      gte(attendance.date, thirtyDaysAgo()))),
    db.select().from(leaveRequests).where(and(
      eq(leaveRequests.userId, profiles.userId),
      eq(leaveRequests.status, 'pending'))),
    // role fit would be computed separately
  ]);

  const emp = profile[0];
  let riskScore = 0;
  const factors: string[] = [];

  // ── Tenure factor ──────────────────────────────────────
  const tenureYears = differenceInYears(new Date(), new Date(emp.hireDate!));
  if (tenureYears < 1) { riskScore += 35; factors.push('New employee (< 1 year)'); }
  else if (tenureYears < 2) { riskScore += 10; factors.push('Short tenure'); }

  // ── Attendance factor ──────────────────────────────────
  const lateCount   = attendanceRows.filter(a => a.status === 'late').length;
  const absentCount = attendanceRows.filter(a => a.status === 'absent').length;
  const absenceRate = Math.round((absentCount / 22) * 100);

  if (absenceRate >= 15) { riskScore += 20; factors.push('High absence rate'); }
  else if (absenceRate >= 10) { riskScore += 12; factors.push('Elevated absence rate'); }
  if (lateCount >= 6) { riskScore += 8; factors.push('Frequent late arrivals'); }

  // ── Leave request frequency ────────────────────────────
```

```
  if (leaveRows.length >= 5) { riskScore += 10; factors.push('Frequent leave requests'); }
  else if (leaveRows.length >= 3) riskScore += 5;

  riskScore = Math.min(riskScore, 100);
  const riskLevel = riskScore >= 70 ? 'critical'
                  : riskScore >= 50 ? 'high'
                  : riskScore >= 30 ? 'medium' : 'low';

  // Persist to cache table
  await db.insert(turnoverRiskCache)
    .values({ profileId, riskScore, riskLevel, factors })
    .onConflictDoUpdate({ target: turnoverRiskCache.profileId,
                          set: { riskScore, riskLevel, factors, computedAt: new Date() } });

  return { riskScore, riskLevel, factors };
}
```

## 7.3 Replacement Candidates (src/scoring/replacements.ts)

```
// GET /api/analytics/replacements?departingProfileId=<uuid>
export async function findReplacementCandidates(departingProfileId: string) {
  const [departing] = await db.select().from(profiles)
    .where(eq(profiles.id, departingProfileId));

  // Find the role definition
  const [role] = await db.select().from(jobRoles)
    .where(eq(jobRoles.title, departing.position ?? ''));
  if (!role) return [];

  // Get all other employees
  const allProfiles = await db.select().from(profiles)
    .where(ne(profiles.id, departingProfileId));

  // Score each candidate (parallel)
  const scored = await Promise.all(allProfiles.map(async p => ({
    profile: p,
    fitScore: await calculateRoleFit(p.id, role.id),
  })));

  return scored.sort((a, b) => b.fitScore - a.fitScore).slice(0, 10);
}
```

## 7.4 Org-Level Skill Gap Analysis

```
// GET /api/analytics/skill-gaps
export async function getOrgSkillGaps() {
  // How many roles demand each skill
  const demand = await db
    .select({ skillId: roleRequiredSkills.skillId,
              demandCount: count() })
    .from(roleRequiredSkills)
    .groupBy(roleRequiredSkills.skillId);

  // Average proficiency across all employees per skill
  const supply = await db
    .select({ skillId: employeeSkills.skillId,
              avgProficiency: avg(employeeSkills.proficiency),
```

```
              headcount: count() })
    .from(employeeSkills)
    .groupBy(employeeSkills.skillId);

  return demand.map(d => ({
    skillId: d.skillId,
    demandCount: d.demandCount,
    ...supply.find(s => s.skillId === d.skillId) ?? { avgProficiency: 0, headcount: 0 },
  })).sort((a, b) => b.demandCount - a.demandCount);
}
```

# 8. Background Jobs (BullMQ)

## 8.1 Queue Setup (src/jobs/queue.ts)

```
import { Queue, Worker } from 'bullmq';
import IORedis from 'ioredis';

const connection = new IORedis(process.env.REDIS_URL!, { maxRetriesPerRequest: null });

export const payrollQueue     = new Queue('payroll',     { connection });
export const notifQueue       = new Queue('notifications', { connection });
export const riskRefreshQueue = new Queue('risk-refresh',  { connection });
```

## 8.2 Payroll Worker (src/jobs/payrollWorker.ts)

```
new Worker('payroll', async job => {
  const { month, year } = job.data;
  const allEmployees = await db.select().from(profiles);

  for (const emp of allEmployees) {
    // Calculate deductions (attendance-based)
    const absentDays = await getAbsentDays(emp.userId, month, year);
    const dailyRate = Number(emp.salary) / 22;
    const deductions = absentDays * dailyRate;
    const allowances = Number(emp.salary) * 0.1; // 10% housing
    const netSalary = Number(emp.salary) + allowances - deductions;

    await db.insert(payroll).values({
      userId: emp.userId, month, year,
      basicSalary: emp.salary!,
      allowances: String(allowances),
      deductions: String(deductions),
      netSalary: String(netSalary),
      status: 'pending',
    }).onConflictDoNothing();

    // Notify employee
    await notifQueue.add('payslip-ready', {
      userId: emp.userId,
      title: `Payslip for ${month}/${year} is ready`,
      message: `Your net salary is ${netSalary.toFixed(2)} EGP.`,
    });
  }
```

```
}, { connection });
```

### 8.3 Notification Worker

```
new Worker('notifications', async job => {
  const { userId, title, message } = job.data;
  await db.insert(notifications).values({ userId, title, message });
  // Supabase Realtime automatically broadcasts INSERT to subscribed clients
}, { connection });
```

### 8.4 Scheduled Risk Refresh

```
// Re-compute turnover risk scores every night at 2am
import cron from 'node-cron';

cron.schedule('0 2 * * *', async () => {
  const allProfiles = await db.select({ id: profiles.id }).from(profiles);
  for (const p of allProfiles) {
    await riskRefreshQueue.add('recompute', { profileId: p.id });
  }
});
```

## 9. Seeding Real Data from mockData.ts

The existing mockData.ts becomes a one-time seed source. Run the seed script to populate the empty PostgreSQL database with the 50 employees, 28 skills, 22 roles, and all related data.

```
// src/db/seed.ts  — run once with: npx tsx src/db/seed.ts
import { db } from '../config/database';
import { skills as skillsData, roles as rolesData,
         employees as employeesData, learningItems, skillChains,
         holidays as holidaysData } from '../../frontend/src/data/mockData';

async function seed() {
  console.log('Seeding skills...');
  await db.insert(skills).values(skillsData.map(s => ({
    id: s.id, name: s.name, category: s.category,
    description: s.description, relatedSkills: s.relatedSkills,
  }))).onConflictDoNothing();

  console.log('Seeding job roles...');
  for (const role of rolesData) {
    const [inserted] = await db.insert(jobRoles).values({
      title: role.title, department: role.department,
      level: role.level, description: role.description,
    }).returning();
    await db.insert(roleRequiredSkills).values(
      role.requiredSkills.map(rs => ({
        roleId: inserted.id, skillId: rs.skillId,
        minProficiency: rs.minProficiency,
      }))
    ).onConflictDoNothing();
  }
```

```
  console.log('Seeding employees...');
  for (const emp of employeesData) {
    // Create Supabase auth user first (via service role)
    const { data: authUser } = await supabaseAdmin.auth.admin.createUser({
      email: emp.email, password: 'SkillSync2024!',
      user_metadata: { full_name: emp.name },
    });
    const userId = authUser.user!.id;

    const [profile] = await db.insert(profiles).values({
      userId, fullName: emp.name, email: emp.email,
      phone: emp.phone, department: emp.department,
      position: emp.currentRole, salary: String(emp.salary ?? 0),
      hireDate: emp.joinDate,
    }).returning();

    await db.insert(employeeSkills).values(
      emp.skills.map(s => ({
        profileId: profile.id, skillId: s.skillId,
        proficiency: s.proficiency,
        lastAssessed: new Date(s.lastAssessed),
      }))
    );

    // Assign leave balances
    await db.insert(leaveBalances).values([
      { userId, leaveType: 'annual', totalDays: 21, usedDays: 0, year: 2025 },
      { userId, leaveType: 'sick',   totalDays: 10, usedDays: 0, year: 2025 },
      { userId, leaveType: 'compassionate', totalDays: 5, usedDays: 0, year: 2025 },
    ]);
  }

  console.log('Seeding holidays...');
  await db.insert(holidays).values(holidaysData).onConflictDoNothing();

  console.log('Seed complete!');
}
seed().catch(console.error);
```

# 10. Connecting the React Frontend

Replace all mockData.ts imports with API calls. Use TanStack Query (already installed) for server state, caching, and background refetching.

## 10.1 API Client (src/lib/api.ts)

```
import { supabase } from './supabase';

const BASE_URL = import.meta.env.VITE_API_URL; // http://localhost:4000

async function apiRequest<T>(path: string, options: RequestInit = {}): Promise<T> {
  const { data: { session } } = await supabase.auth.getSession();
  const token = session?.access_token ?? '';
```

```
  const res = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
      ...options.headers,
    },
  });

  if (!res.ok) {
    const err = await res.json();
    throw new Error(err.error ?? 'API error');
  }
  return res.json();
}

export const api = {
  get:    <T>(path: string) => apiRequest<T>(path),
  post:   <T>(path: string, body: unknown) =>
    apiRequest<T>(path, { method: 'POST', body: JSON.stringify(body) }),
  patch:  <T>(path: string, body: unknown) =>
    apiRequest<T>(path, { method: 'PATCH', body: JSON.stringify(body) }),
  delete: <T>(path: string) => apiRequest<T>(path, { method: 'DELETE' }),
};
```

## 10.2 TanStack Query Hooks (src/hooks/useEmployees.ts)

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { api } from '../lib/api';

// Replace:  import { employees } from '@/data/mockData';
// With:     const { data: employees } = useEmployees();

export function useEmployees(params?: { department?: string; search?: string }) {
  const query = new URLSearchParams(params as any).toString();
  return useQuery({
    queryKey: ['employees', params],
    queryFn:  () => api.get<Employee[]>(`/api/employees?${query}`),
    staleTime: 1000 * 60 * 5,  // 5-minute cache
  });
}

export function useMyProfile() {
  return useQuery({
    queryKey: ['profile', 'me'],
    queryFn:  () => api.get('/api/employees/me'),
  });
}

export function useUpdateProfile() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (data: Partial<Profile>) => api.patch('/api/employees/me', data),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['profile', 'me'] }),
  });
}

// Similarly for skills, attendance, leaves, payroll, turnover risk, etc.
```

### 10.3 What to Remove from the Frontend

```
REMOVE: src/data/mockData.ts — replace all imports with API hooks
REMOVE: src/contexts/EmployeeContext.tsx — replaced by useMyProfile() hook
REMOVE: All inline scoring logic in HRTurnoverPrediction.tsx, ManagerDashboard.tsx, etc.
REMOVE: generateTurnoverRiskData() — now served by GET /api/analytics/turnover
REMOVE: findReplacementCandidates() — now served by GET /api/analytics/replacements
ADD:    VITE_API_URL=http://localhost:4000 to frontend .env
```

### 10.4 Real-time Notifications

```
// src/hooks/useNotifications.ts
export function useNotifications() {
  const qc = useQueryClient();

  useEffect(() => {
    const { data: { subscription } } = supabase.auth.onAuthStateChange((_, session) => {
      if (!session) return;
      // Subscribe to new notifications for this user
      const channel = supabase
        .channel('notifications')
        .on('postgres_changes', {
          event: 'INSERT',
          schema: 'public',
          table: 'notifications',
          filter: `user_id=eq.${session.user.id}`,
        }, () => {
          qc.invalidateQueries({ queryKey: ['notifications'] });
        })
        .subscribe();
      return () => supabase.removeChannel(channel);
    });
    return () => subscription.unsubscribe();
  }, []);

  return useQuery({
    queryKey: ['notifications'],
    queryFn:  () => api.get('/api/notifications'),
  });
}
```

## 11. Security Implementation

### 11.1 Row-Level Security in Supabase

Enable RLS on all tables and create policies so employees can only read their own rows:

```
-- Enable RLS on every table
ALTER TABLE attendance         ENABLE ROW LEVEL SECURITY;
ALTER TABLE leave_requests     ENABLE ROW LEVEL SECURITY;
ALTER TABLE payroll            ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE todos              ENABLE ROW LEVEL SECURITY;
ALTER TABLE notifications      ENABLE ROW LEVEL SECURITY;

-- Employees: own rows only
CREATE POLICY emp_own_attendance ON attendance
  FOR ALL USING (user_id = auth.uid());

-- HR Admins: all rows
CREATE POLICY hr_all_attendance ON attendance
  FOR ALL USING (
    EXISTS (SELECT 1 FROM user_roles
            WHERE user_id = auth.uid() AND role = 'hr_admin')
  );

-- Managers: own department rows
CREATE POLICY mgr_dept_leaves ON leave_requests
  FOR SELECT USING (
    user_id IN (
      SELECT p.user_id FROM profiles p
      WHERE p.department = (SELECT department FROM profiles WHERE user_id = auth.uid())
    )
  );
```

## 11.2 Express Security Middleware (src/app.ts)

```
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';

const app = express();

app.use(helmet());  // Security headers
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
app.use(express.json({ limit: '1mb' }));

// Rate limiting: 100 requests per 15 minutes per IP
app.use('/api', rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));

// Stricter limit for auth endpoints
app.use('/api/auth', rateLimit({ windowMs: 60 * 1000, max: 10 }));
```

## 11.3 Audit Logging Middleware

```
// Automatically log all state-changing requests
export async function auditLog(req: AuthRequest, res: Response, next: NextFunction) {
  const originalJson = res.json.bind(res);
  res.json = function(body) {
    if (['POST','PATCH','DELETE'].includes(req.method) && res.statusCode < 400) {
      db.insert(auditLogs).values({
        userId:     req.user?.id,
        action:     req.method,
        entityType: req.baseUrl.split('/').pop() ?? 'unknown',
        newValues:  body,
        ipAddress:  req.ip,
      }).catch(console.error);
    }
```

```
    return originalJson(body);
  };
  next();
}
```

# 12. Deployment

## 12.1 Production Build

```
# Build TypeScript
npm run build  # tsc → dist/

# Start production server
NODE_ENV=production node dist/index.js

# Or with PM2 for process management
npm install -g pm2
pm2 start dist/index.js --name skillsync-api
pm2 startup && pm2 save
```

## 12.2 Deployment Options

| Platform | Notes |
|---|---|
| Railway.app | Easiest — connect GitHub repo, set env vars, auto-deploys. Free tier available. |
| Render.com | Similar to Railway. Supports background workers as separate services. |
| Fly.io | Docker-based, global edge deployment. Good for low latency. |
| AWS EC2 | Full control. Set up with Nginx reverse proxy + PM2 + Let's Encrypt SSL. |
| Supabase Edge Functions | For simple endpoints only — no BullMQ support. |

## 12.3 Docker Compose (Local Dev)

```
version: '3.9'
services:
  api:
    build: .
    ports: ['4000:4000']
    environment:
      - DATABASE_URL=postgresql://postgres:password@postgres:5432/skillsync
      - REDIS_URL=redis://redis:6379
    depends_on: [postgres, redis]
    volumes: ['./src:/app/src']   # hot reload
  postgres:
    image: postgres:15
    environment: { POSTGRES_DB: skillsync, POSTGRES_PASSWORD: password }
    volumes: ['pgdata:/var/lib/postgresql/data']
    ports: ['5432:5432']
  redis:
```

```
    image: redis:7-alpine
    ports: ['6379:6379']
volumes:
  pgdata:
```

## 13. Step-by-Step Implementation Checklist

### Phase 1 — Database & Schema (Week 1)

1. Create Supabase project, copy DATABASE_URL and SUPABASE_JWT_SECRET
2. Run `npm init` in skillsync-api/, install all dependencies from Section 3
3. Define all Drizzle schema tables (Section 4)
4. Run `npx drizzle-kit push:pg` to create tables in Supabase
5. Enable RLS on all tables (Section 11.1)
6. Run seed script to import mock data (Section 9)

### Phase 2 — Auth & Core Middleware (Week 1)

7. Implement verifyToken middleware (Section 5.1)
8. Implement authorize() middleware (Section 5.2)
9. Wire up helmet, cors, rate limiting (Section 11.2)
10. Add audit logging middleware (Section 11.3)
11. Add global error handler

### Phase 3 — Feature Modules (Weeks 2–3)

12. Employees module — list, get, create, update, me
13. Skills module — list skills, update own proficiency, org gaps
14. Attendance module — check-in, check-out, history
15. Leave module — submit, approve/reject, balance
16. Payroll module — view own, HR view all, run job
17. Todos module — full CRUD for own todos
18. Notifications module — list own, mark read
19. Resignations module — submit, HR review
20. Holidays module — CRUD for HR, view for all
21. Departments & Roles modules — CRUD for HR
22. Audit module — HR read-only log viewer

### Phase 4 — Analytics & Scoring (Week 3)

23. Move calculateRoleFit() to src/scoring/roleFit.ts (Section 7.1)
24. Move turnover risk to src/scoring/turnoverRisk.ts (Section 7.2)

25. Move replacement candidates to src/scoring/replacements.ts (Section 7.3)
26. Move org skill gaps to src/scoring/skillGaps.ts (Section 7.4)
27. Set up nightly cron for risk score refresh (Section 8.4)

## Phase 5 — Real-time & Chat (Week 4)

28. Set up BullMQ queues and Redis connection (Section 8.1)
29. Implement payroll background worker (Section 8.2)
30. Implement notification worker (Section 8.3)
31. Set up Supabase Realtime for chat_messages (Section 6.6)
32. Enable Realtime on notifications table in Supabase dashboard

## Phase 6 — Frontend Migration (Weeks 4–5)

33. Create src/lib/api.ts with token-injecting fetch wrapper (Section 10.1)
34. Create TanStack Query hooks for every domain (Section 10.2)
35. Replace all mockData.ts imports page by page
36. Remove EmployeeContext.tsx — use useMyProfile() hook
37. Implement useNotifications() with Realtime subscription
38. Remove all inline scoring functions from React components
39. Update .env with VITE_API_URL

## Phase 7 — Testing & Deployment (Week 5)

40. Write integration tests with Vitest + Supertest for every route
41. Verify RLS policies block cross-user data access
42. Load test payroll run job with 50 employees
43. Deploy API to Railway or Render (Section 12)
44. Deploy frontend to Vercel or Netlify
45. Configure custom domain, SSL, CORS origin